Министерство образования Республики Беларусь
г. Минск
Государственное учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Лабораторная работа № 4

«Многомерные массивы»
по дисциплине «Основы алгоритмизации и программирования»
Вариант 5

Учебная группа 658304

Выполнил: Дерновский А. П.
Проверил: Селезнев А. И.

Минск 2026

Многомерные массивы

Лабораторная работа № 4

Дерновский А. П.
Основы алгоритмизации и программирования, группа 658304
вариант 5 · Минск, 2026

Цель работы

Научиться разрабатывать алгоритмы для работы с многомерными массивами и писать код на языке Си по составленному алгоритму. Лабораторная работа включает в себя 3 задачи для выполнения. Задачи решаются последовательно.

Во всех трёх программах матрица объявлена статически внутри `main`: `int a[MAX_N][MAX_N]` при `MAX_N = 100`. Место выделяется сразу под 100×100 элементов, а фактические размеры n и m вводит пользователь; они проверяются в диапазоне от 1 до 100. Глобальных массивов нет — матрица передаётся в функции параметром. Заполнение выбирается пользователем: ручной ввод (каждый элемент отдельной строкой) или случайные числа от -50 до 50. Введённую матрицу выводит отдельная функция `print_matrix`, она вызывается сразу после заполнения. Ввод каждой величины идёт через `fgets` и `sscanf`: строка читается целиком, поэтому остаток ошибочного ввода не превращается в следующий элемент, а при неверном значении запрос повторяется. На блок-схемах блоки проверки ввода опущены, чтобы не перегружать лист; полная проверка показана в листингах.

Задание № 1

Описание задания. Дана целочисленная прямоугольная матрица. Определить количество столбцов, не содержащих ни одного нулевого элемента.

Ход решения. Размеры матрицы читает `read_size`: два числа разбираются из одной строки и проверяются на диапазон 1..100. Затем `fill_matrix` спрашивает режим заполнения и заполняет матрицу вручную или случайными числами. Каждый элемент при ручном вводе читает `read_int`; он же используется для размеров и режима, поэтому проверка ввода у всех величин одинаковая.

Столбцы перебирает `count_cols_without_zero`. Внешний цикл идёт по столбцам, внутренний — по строкам; флаг `has_zero` сбрасывается перед каждым столбцом и устанавливается, как только в нём встретился ноль. После внутреннего цикла флаг всё ещё равен нулю только тогда, когда нулевых элементов в столбце нет, — такой столбец добавляется к счётчику. Ноль — единственное значение, которое проверяется, поэтому один проход по матрице даёт $O(nm)$ операций; повторный просмотр не нужен.

В ручном запуске матрица 3×4 имела вид

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 0 & 6 & 7 \\ 8 & 9 & 0 & 10 \end{pmatrix}$$

Нули стоят во втором и третьем столбцах, поэтому столбцов без нулевых элементов два — первый и четвёртый. В случайном запуске 3×3 нулей не оказалось вовсе, и счётчик напечатал 3. Первый запуск также показывает работу проверки ввода: строка `abc` отвергнута как размер матрицы, после чего введены корректные `3 4`.

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// Задача 1. Вариант 5
// Дана целочисленная прямоугольная матрица. Определить количество
// столбцов, не содержащих ни одного нулевого элемента.

#define MAX_N 100
#define RND_MIN (-50)
#define RND_MAX 50

// Читает строку и разбирает из неё одно целое число в диапазоне [lo, hi].
// Возвращает 0, если строки кончились (конец файла).
static int read_int(const char *prompt, const int lo, const int hi, int *value)
{
    char line[128];
    int v;

    for (;;) {
        printf("%s", prompt);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d", &v) == 1 && v >= lo && v <= hi) {
            *value = v;
            return 1;
        }
        printf("Ошибка: нужно целое число от %d до %d.\n", lo, hi);
    }
}

// Размеры прямоугольной матрицы: два числа в одной строке.
static int read_size(int *n, int *m)
{
    char line[128];

    for (;;) {
        printf("Размеры матрицы n m (1..%d): ", MAX_N);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d %d", n, m) == 2 &&
            *n >= 1 && *n <= MAX_N && *m >= 1 && *m <= MAX_N) {
            return 1;
        }
        printf("Ошибка: нужно два целых числа от 1 до %d.\n", MAX_N);
    }
}

// Заполнение матрицы: 1 — вручную, 2 — случайными числами.
static int fill_matrix(int a[][MAX_N], const int n, const int m)
{
    char prompt[32];
    int mode, i, j;

    if (!read_int("Заполнение: 1 - вручную, 2 - случайно: ", 1, 2, &mode)) {
        return 0;
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (mode == 1) {
                snprintf(prompt, sizeof prompt, "a[%d][%d] = ", i, j);
                if (!read_int(prompt, INT_MIN, INT_MAX, &a[i][j])) {
                    return 0;
                }
            } else {
                a[i][j] = rand() % (RND_MAX - RND_MIN + 1) + RND_MIN;
            }
        }
    }
    return 1;
}

static void print_matrix(const char *title, const int a[][MAX_N],
                         const int n, const int m)
{
    int i, j;

```

```c
    printf("%s\n", title);
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            printf("%5d", a[i][j]);
        }
        printf("\n");
    }
}

// Считает столбцы, в которых нет ни одного нуля.
static int count_cols_without_zero(const int a[][MAX_N],
                                   const int n, const int m)
{
    int count, i, j, has_zero;

    count = 0;
    for (j = 0; j < m; j++) {
        has_zero = 0;
        for (i = 0; i < n; i++) {
            if (a[i][j] == 0) {
                has_zero = 1;
                break;
            }
        }
        if (!has_zero) {
            count++;
        }
    }
    return count;
}

int main(void)
{
    int a[MAX_N][MAX_N];
    int n, m, count;

    if (!read_size(&n, &m)) {
        return 0;
    }
    if (!fill_matrix(a, n, m)) {
        return 0;
    }

    print_matrix("Введённая матрица:", a, n, m);

    count = count_cols_without_zero(a, n, m);
    printf("Столбцов без нулевых элементов: %d\n", count);

    return 0;
}
```

Листинг 1. Программа к заданию № 1

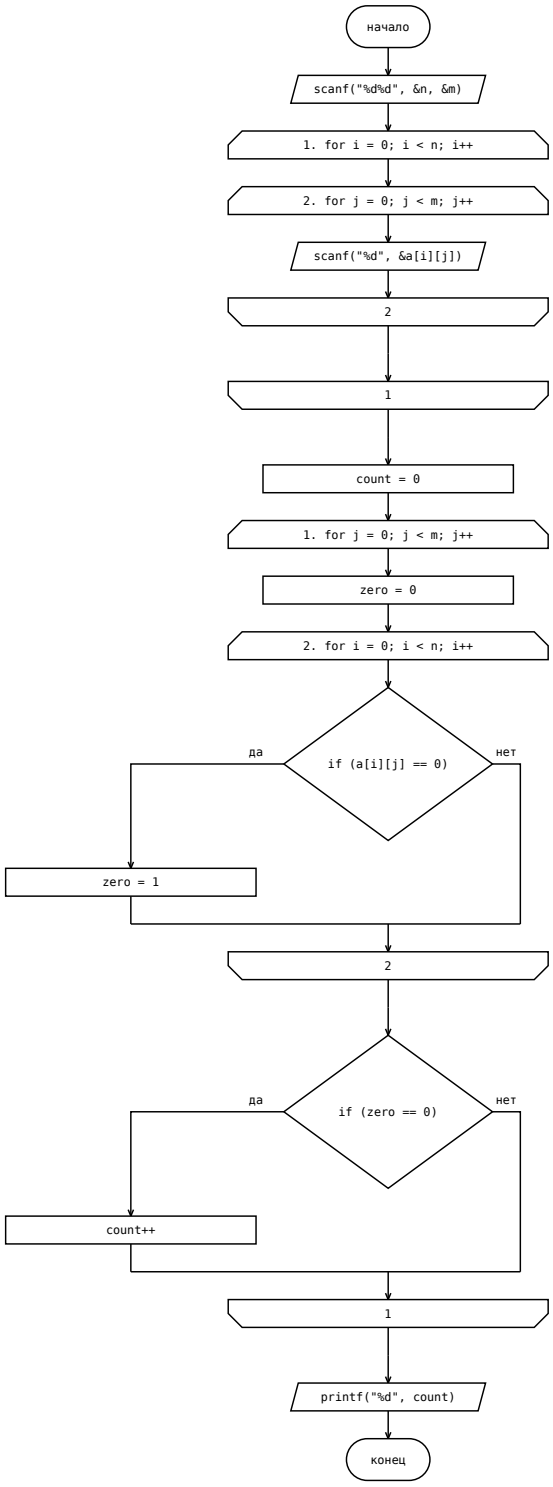


Рис. 1. Блок-схема программы к заданию № 1

```
sehaxe@cachyos: ~/OAiP-Laba4/home

sehaxe@cachyos: /OAiP-Laba4/home$ ./1/main
Размеры матрицы n m (1..100): abc
Ошибка: нужно два целых числа от 1 до 100.
Размеры матрицы n m (1..100): 3 4
Заполнение: 1 - вручную, 2 - случайно: 1
a[0][0] = 1
a[0][1] = 2
a[0][2] = 3
a[0][3] = 4
a[1][0] = 5
a[1][1] = 0
a[1][2] = 6
a[1][3] = 7
a[2][0] = 8
a[2][1] = 9
a[2][2] = 0
a[2][3] = 10
Введённая матрица:
    1    2    3    4
    5    0    6    7
    8    9    0   10
Столбцов без нулевых элементов: 2
sehaxe@cachyos: /OAiP-Laba4/home$ ./1/main
Размеры матрицы n m (1..100): 3 3
Заполнение: 1 - вручную, 2 - случайно: 2
Введённая матрица:
  -18  -18    4
  -38    2    6
  -42  -20   -6
Столбцов без нулевых элементов: 3
```

Рис. 2. Результаты выполнения задания № 1

Задание № 2

Описание задания. Проверить, есть ли в матрице хотя бы одна строка, содержащая положительный элемент, и найти её номер. Знаки элементов предыдущей строки изменить на противоположные.

Ход решения. Номер строки ищет `first_row_with_positive`: внешний цикл идёт по строкам, внутренний — по столбцам, и при первом же положительном элементе функция сразу возвращает индекс строки. Если положительных элементов нет ни в одной строке, возвращается -1 и программа сообщает, что таких строк нет.

Найденная строка печатается с номером, начиная с единицы, и индексом Си в скобках. Предыдущая строка существует только при индексе больше нуля; если первая же строка содержит положительный элемент, менять нечего, и программа сообщает об этом. В остальных случаях `flip_row` проходит по предыдущей строке и меняет знак каждого

элемента унарным минусом; затем матрица печатается ещё раз, чтобы изменение было видно.

В ручном запуске матрица 4×3 была такой:

$$\begin{pmatrix} -1 & -2 & -3 \\ 4 & 5 & -6 \\ 0 & 1 & 2 \\ -7 & -8 & -9 \end{pmatrix}$$

Первая строка с положительным элементом — вторая (индекс 1): в ней стоят 4 и 5. Предыдущая, первая строка меняет знаки: −1, −2, −3 превращаются в 1, 2, 3. Во втором запуске положительный элемент стоит уже в первой строке, поэтому предыдущей строки нет и матрица остаётся без изменений.

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// Задача 2. Вариант 5
// Проверить, есть ли в матрице хотя бы одна строка, содержащая
// положительный элемент, и найти её номер. Знаки элементов предыдущей
// строки изменить на противоположные.

#define MAX_N 100
#define RND_MIN (-50)
#define RND_MAX 50

static int read_int(const char *prompt, const int lo, const int hi, int *value)
{
    char line[128];
    int v;

    for (;;) {
        printf("%s", prompt);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d", &v) == 1 && v >= lo && v <= hi) {
            *value = v;
            return 1;
        }
        printf("Ошибка: нужно целое число от %d до %d.\n", lo, hi);
    }
}

static int read_size(int *n, int *m)
{
    char line[128];

    for (;;) {
        printf("Размеры матрицы n m (1..%d): ", MAX_N);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d %d", n, m) == 2 &&
            *n >= 1 && *n <= MAX_N && *m >= 1 && *m <= MAX_N) {
            return 1;
        }
        printf("Ошибка: нужно два целых числа от 1 до %d.\n", MAX_N);
    }
}

static int fill_matrix(int a[][MAX_N], const int n, const int m)
{
    char prompt[32];
    int mode, i, j;

    if (!read_int("Заполнение: 1 - вручную, 2 - случайно: ", 1, 2, &mode)) {
        return 0;
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (mode == 1) {
                snprintf(prompt, sizeof prompt, "a[%d][%d] = ", i, j);
                if (!read_int(prompt, INT_MIN, INT_MAX, &a[i][j])) {
                    return 0;
                }
            } else {
                a[i][j] = rand() % (RND_MAX - RND_MIN + 1) + RND_MIN;
            }
```

```c
        }
    }
    return 1;
}

static void print_matrix(const char *title, const int a[][MAX_N],
                         const int n, const int m)
{
    int i, j;

    printf("%s\n", title);
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            printf("%5d", a[i][j]);
        }
        printf("\n");
    }
}

// Номер первой строки с положительным элементом или -1, если такой нет.
static int first_row_with_positive(const int a[][MAX_N],
                                   const int n, const int m)
{
    int i, j;

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (a[i][j] > 0) {
                return i;
            }
        }
    }
    return -1;
}

static void flip_row(int a[][MAX_N], const int m, const int row)
{
    int j;

    for (j = 0; j < m; j++) {
        a[row][j] = -a[row][j];
    }
}

int main(void)
{
    int a[MAX_N][MAX_N];
    int n, m, row;

    if (!read_size(&n, &m)) {
        return 0;
    }
    if (!fill_matrix(a, n, m)) {
        return 0;
    }

    print_matrix("Введённая матрица:", a, n, m);

    row = first_row_with_positive(a, n, m);
    if (row == -1) {
        printf("Строк с положительным элементом нет.\n");
    } else {
        printf("Первая строка с положительным элементом: %d (индекс %d)\n",
               row + 1, row);
        if (row == 0) {
            printf("Предыдущей строки нет, матрица не изменилась.\n");
        } else {
            flip_row(a, m, row - 1);
            printf("Знаки элементов строки %d изменены на противоположные.\n",
                   row);
            print_matrix("Матрица после изменения:", a, n, m);
        }
    }

    return 0;
}

```

Листинг 2. Программа к заданию № 2

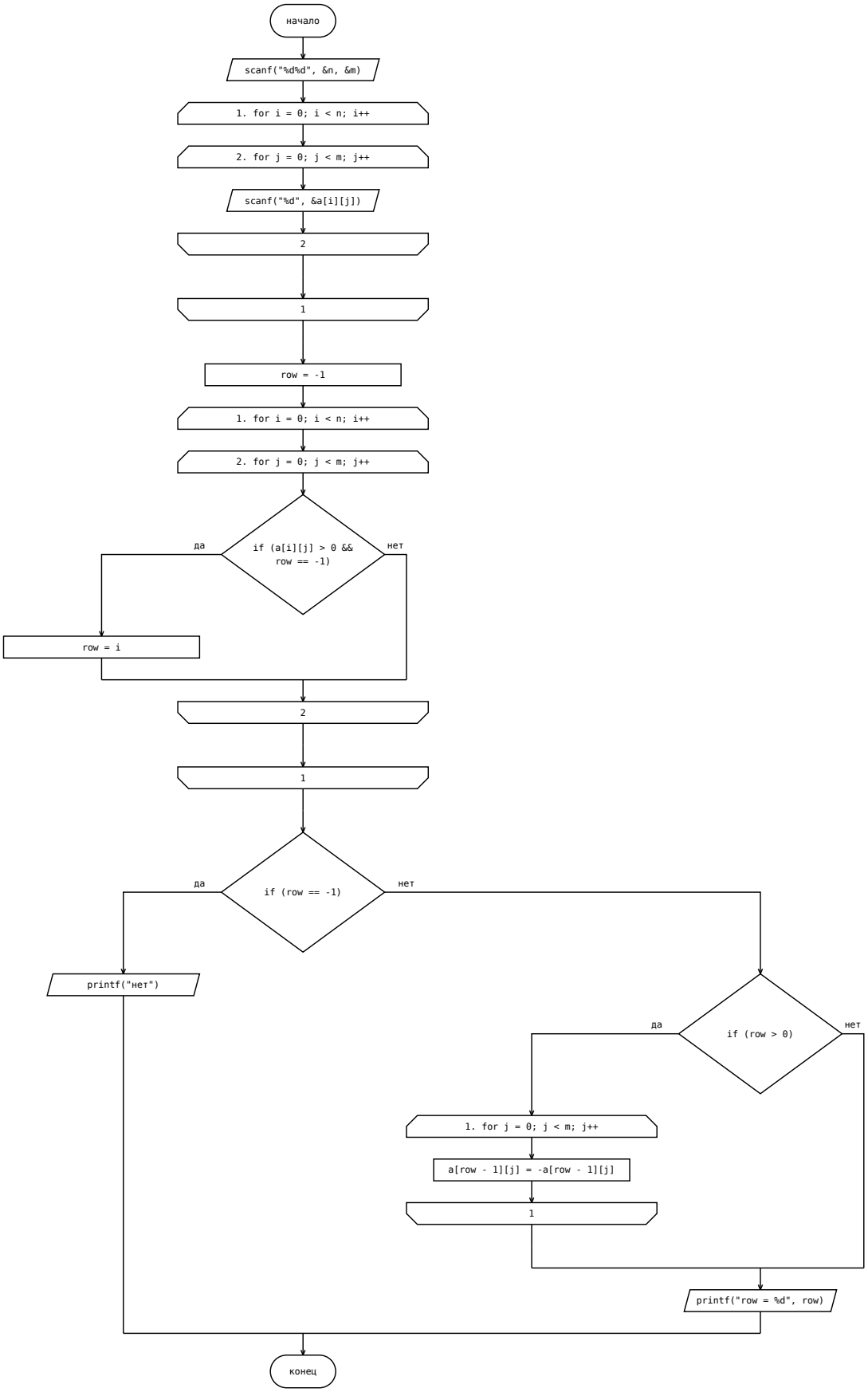


Рис. 3. Блок-схема программы к заданию № 2

```
sehaxe@cachyos: ~/OAiP-Laba4/home

sehaxe@cachyos: /OAiP-Laba4/home$ ./2/main
Размеры матрицы n m (1..100): 4 3
Заполнение: 1 - вручную, 2 - случайно: 1
a[0][0] = -1
a[0][1] = -2
a[0][2] = -3
a[1][0] = 4
a[1][1] = 5
a[1][2] = -6
a[2][0] = 0
a[2][1] = 1
a[2][2] = 2
a[3][0] = -7
a[3][1] = -8
a[3][2] = -9
Введённая матрица:
   -1   -2   -3
    4    5   -6
    0    1    2
   -7   -8   -9
Первая строка с положительным элементом: 2 (индекс 1)
Знаки элементов строки 1 изменены на противоположные.
Матрица после изменения:
    1    2    3
    4    5   -6
    0    1    2
   -7   -8   -9
sehaxe@cachyos: /OAiP-Laba4/home$ ./2/main
Размеры матрицы n m (1..100): 2 2
Заполнение: 1 - вручную, 2 - случайно: 1
a[0][0] = 7
a[0][1] = 8
a[1][0] = -5
a[1][1] = -6
Введённая матрица:
    7    8
   -5   -6
Первая строка с положительным элементом: 1 (индекс 0)
Предыдущей строки нет, матрица не изменилась.
```

Рис. 4. Результаты выполнения задания № 2

Задание № 3

Описание задания. В квадратной матрице размером N×N найти максимальный элемент в 1-ой области.

Ход решения. Область 1 — верхний треугольник квадратной матрицы, ограниченный главной и побочной диагоналями (верхний сектор на рисунке методички). Для элемента с индексами i и j это означает одновременно $j > i$ (выше главной диагонали) и $j < n - 1 -$

i (выше побочной). Оба условия записаны прямо в заголовке внутреннего цикла `for (j = i + 1; j < n - 1 - i; j++)`, поэтому перебираются только элементы области, без проверок внутри тела.

Максимум ищет `max_region1`. Первый найденный элемент запоминается безусловно (флаг `found`), а каждый следующий сравнивается с текущим максимумом. Если область пуста — при $n < 3$ между диагоналями нет ни одной ячейки, — функция возвращает `found = 0`, и программа печатает сообщение вместо значения.

В ручном запуске матрица 5×5 заполнена числами от 1 до 25 по строкам. В области 1 лежат ячейки (1,2), (1,3), (1,4) и (2,3) в нумерации с единицы, то есть значения 2, 3, 4 и 8; максимальное из них — 8. В случайном запуске 6×6 в область попали −18, 4, −38, 2, −6 и 44, поэтому напечатано 44.

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// Задача 3. Вариант 5
// В квадратной матрице размером NxN найти максимальный элемент
// в 1-ой области (верхний треугольник между главной и побочной
// диагоналями).

#define MAX_N 100
#define RND_MIN (-50)
#define RND_MAX 50

static int read_int(const char *prompt, const int lo, const int hi, int *value)
{
    char line[128];
    int v;

    for (;;) {
        printf("%s", prompt);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d", &v) == 1 && v >= lo && v <= hi) {
            *value = v;
            return 1;
        }
        printf("Ошибка: нужно целое число от %d до %d.\n", lo, hi);
    }
}

// Размер квадратной матрицы.
static int read_size(int *n)
{
    char line[128];

    for (;;) {
        printf("Размер матрицы n (1..%d): ", MAX_N);
        if (fgets(line, sizeof line, stdin) == NULL) {
            return 0;
        }
        if (sscanf(line, "%d", n) == 1 && *n >= 1 && *n <= MAX_N) {
            return 1;
        }
        printf("Ошибка: нужно целое число от 1 до %d.\n", MAX_N);
    }
}

static int fill_matrix(int a[][MAX_N], const int n)
{
    char prompt[32];
    int mode, i, j;

    if (!read_int("Заполнение: 1 - вручную, 2 - случайно: ", 1, 2, &mode)) {
        return 0;
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (mode == 1) {
                snprintf(prompt, sizeof prompt, "a[%d][%d] = ", i, j);
                if (!read_int(prompt, INT_MIN, INT_MAX, &a[i][j])) {
                    return 0;
                }
            } else {
```

```c
                a[i][j] = rand() % (RND_MAX - RND_MIN + 1) + RND_MIN;
            }
        }
    }
    return 1;
}

static void print_matrix(const char *title, const int a[][MAX_N], const int n)
{
    int i, j;

    printf("%s\n", title);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%5d", a[i][j]);
        }
        printf("\n");
    }
}

// Максимум в области 1: элементы строго выше обеих диагоналей,
// то есть j > i и j < n - 1 - i. found = 0, если область пуста.
static int max_region1(const int a[][MAX_N], const int n, int *found)
{
    int best, i, j;

    best = 0;
    *found = 0;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n - 1 - i; j++) {
            if (!*found || a[i][j] > best) {
                best = a[i][j];
                *found = 1;
            }
        }
    }
    return best;
}

int main(void)
{
    int a[MAX_N][MAX_N];
    int n, found, best;

    if (!read_size(&n)) {
        return 0;
    }
    if (!fill_matrix(a, n)) {
        return 0;
    }

    print_matrix("Введённая матрица:", a, n);

    best = max_region1(a, n, &found);
    if (!found) {
        printf("Область 1 пуста: в матрице меньше трёх строк.\n");
    } else {
        printf("Максимальный элемент в области 1: %d\n", best);
    }

    return 0;
}

```

Листинг 3. Программа к заданию № 3

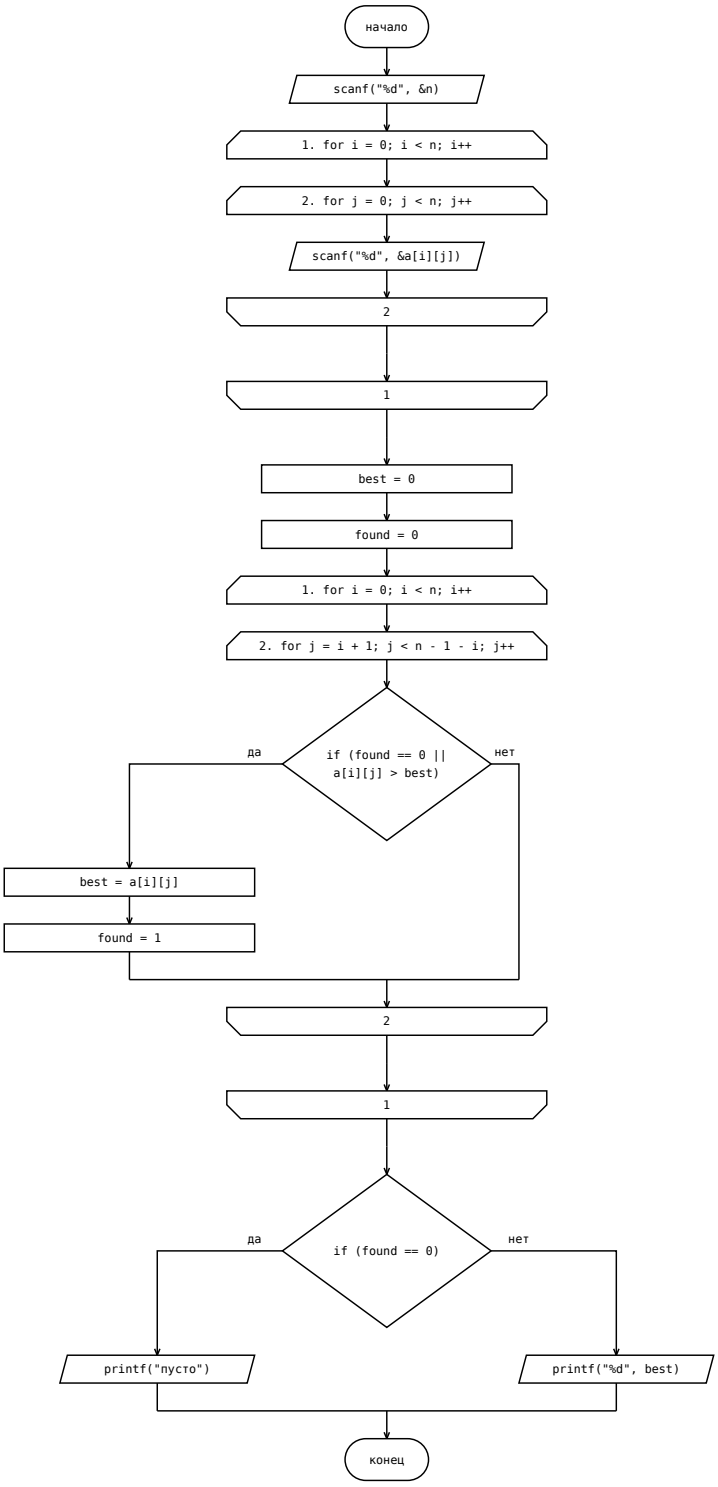


Рис. 5. Блок-схема программы к заданию № 3

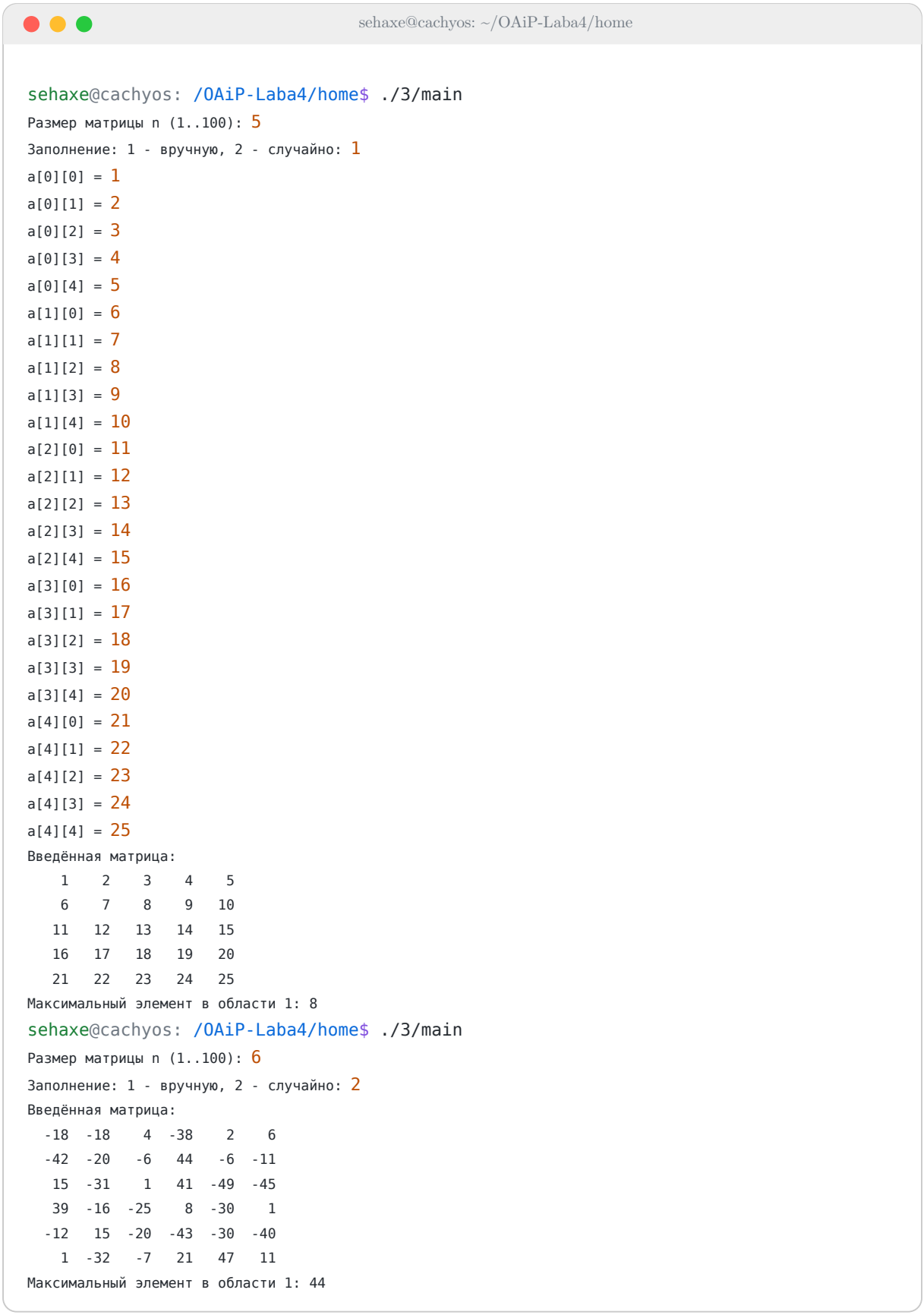


Рис. 6. Результаты выполнения задания № 3